

ASP.NET Web Forms

ASP.NET Web Forms is part of the .NET infrastructure that provides a programming model and various services that are necessary to build up web applications.

The main benefit of ASP.NET Web Forms is the level of abstraction that it provides compared to the request/response approach and the creation of the HTML that is sent to the client.

ViewState – The response to change in the page content is enhanced through the use of ViewState. ViewState is how ASP.NET Web Forms take this customized approach and transfer state information between the browser and the server.

Events and Page Life Cycle

The ASP.NET lifecycle allows the developer to interact with information at various points in the HTML creation phase. As part of the flow, the developer can also use event handlers to respond to events that may happen on the client, including clicking a button or selecting an item in a dropdown list.

ASP.NET Page Lifecycle Stages:

Stages	Description
Page Request	This stage happens before the page-calling process starts. It is when the system determines whether run-time compilation is necessary, whether cached output can be returned, or whether a compiled page needs to be run.
Start	The page starts to do some processing on the HTTP request. Some base variables are initialized, such as Request, Response, and the UICulture.
Initialization	During this phase, the controls on the page are initialized and assigned their unique IDs. Master pages and themes are applied as applicable.
Load	If the request is a postback, control information is loaded with the information recovered from the view state.
Postback Event Handling	If the request is a postback, all the various controls fire their event handlers as needed. Validation also happens at this time.
Rendering	Before the rendering stage starts, ViewState is saved for the page and all of the controls as configured.
Unload	This happens after the content is created and sent to the client. Objects are unloaded from memory and cleanup happens.

ASP.NET Page Lifecycle Events

Event Name	Description
PreInit	Raised after the start stage is complete and before the initialization stage begins. It is used to create or recreate dynamic controls, setting master pages or themes dynamically
Init	This event is raised after all the controls have been initialized. It is used to initialize control properties.
InitComplete	Only one thing happens between Init and InitComplete, and that is the enabling of view state for the controls. Changes applied in this event and after will impact view state to make sure it will be available next postback.
PreLoad	Raised after the page manages the view state information for itself and all controls.
Load	The OnLoad method is called in a page, which then recursively calls that same method on every control.

Control Events	These are specific control-based events, such as the Click of a button, or TextChanged on a text box.
LoadComplete	This event is raised after all the event handling has occurred.
PreRender	After all the controls have been loaded, the Page object starts its Prerender phase.
PreRenderComplete	Raised after every data-bound control has been bound.
SaveStateComplete	Raised after view state and control state have been saved for the page and for all controls.
Render	This is not an event. Rather, at this point in the process, the Page object calls this method on each control.
Unload	This is used to perform special cleanup activities, such as closing files or database connections, logging, etc.

Control Library

ASP.NET has a powerful set of built-in server controls that provides a boost in development speed and enhance rapid application development.

ASP.NET server controls are items that a developer places on an ASP.NET web page. Many of these server controls are similar to HTML elements and others of these server controls allow for more complex behavior such as calendar and controls that are used to connect to data resources.

There are four (4) main types of controls:

1. HTML server controls – wrappers for traditional HTML elements.
2. Web server controls – tends to encompass more functionality and be more abstract than an HTML server control.
3. Validation controls – ensures that the values entered into other controls meet certain criteria, or are valid.
4. User controls – controls that you build yourself. If a set of functionality needs to be available on multiple pages, then it is most likely that you should create this functionality as a user control.

REFERENCES:

Andrew, L. (2021). *ASP.NET Core in action*. Manning Publications Co.

Benjamin J. & Jason M. (2018). *Mastering Bootstrap 4 2nd Edition: Master the latest version of Bootstrap 4 to build highly customized responsive web apps*. Packt Publishing.

De Oliveria, J. & Bruchet, M. (2017). *Learning ASP.NET Core 2.0*. Packt Publishing.

Rippon, C. (2020). *ASP.NET Core 5 and React: Full-stack web development using .NET 5, React 17, and TypeScript 4*. Packt Publishing.

[https://docs.microsoft.com/en-us/previous-versions/aspnet/ms178472\(v=vs.100\)#:~:text=When%20an%20ASP.NET%20page,event%20handler%20code%2C%20and%20rendering.](https://docs.microsoft.com/en-us/previous-versions/aspnet/ms178472(v=vs.100)#:~:text=When%20an%20ASP.NET%20page,event%20handler%20code%2C%20and%20rendering.)